Divisor de frequência em VHDL

1. Introdução:

Em circuitos digitais implementados em lógica configurável como cpld's e fpga's, é bastante comum dispor-se um relógio (clock) master de alta frequência, a partir do qual vários outros sinais de relógio, necessários à operação do circuito, são derivados através de divisores de frequência.

Uma possível implementação será então abordada neste tutorial, inclusive considerando a possibilidade de divisores em cadeia, oferecendo mais de uma frequência na saída.

2. O Circuito básico

A Figura 1 abaixo ilustra um circuito básico usado com divisor de frequência.

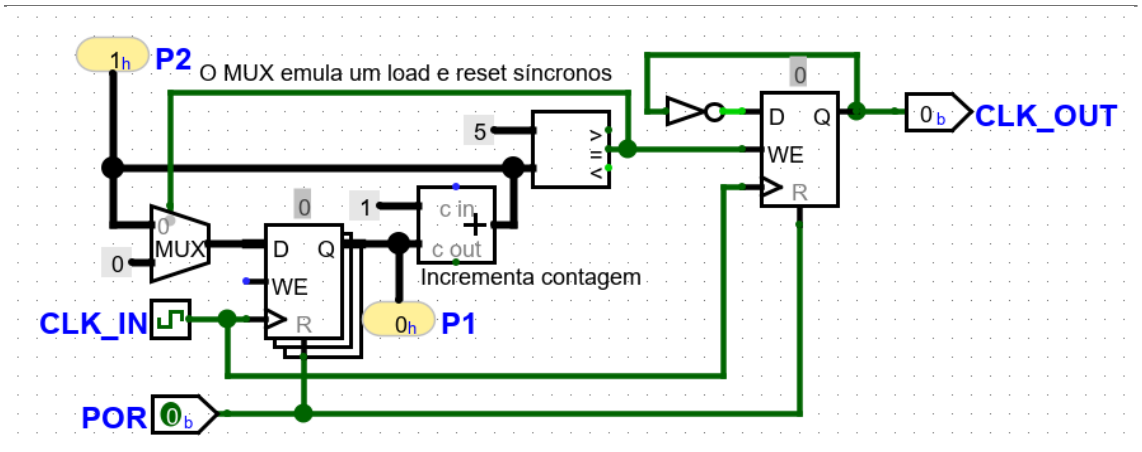

Figura 1: Circuito básico de um divisor de frequência digital síncrono.

O circuito apresentado é totalmente síncrono com CLK_IN.

- O registrador de 3 bits com o somador e o mux formam um contador síncrono.
- Quando a contagem atinge o valor 5 o mux comuta para zero fazendo com que ocorra um reset síncrono.
- A saída do registrador, portanto, conta de 0 a 4 e, a seguir, retorna a zero, implementando um divisor por 5.
- A saída do comparador permanece em alto durante o 5º clock, habilitando a comutação de CLK_OUT na próxima transição do relógio
- Dessa maneira, o sinal de saída CLK_OUT permanece 5 clocks em baixo e 5 em alto, implementando um sinal com 1/10 da frequência de entrada.

No exemplo, foi implementado um divisor por 10 por simplicidade de análise. Na verdade, foi feita uma divisão por 5 e depois por 2, para garantir um "duty cycle" de 50% na saída. Note-se que esse processo só funciona quando o divisor final for par.

Alterando-se a quantidade de bits do registrador, do somador e do comparador, e ajustando-se o valor de comparação, pode-se obter um divisor que qualquer valor.

Por exemplo, para obter um clock de 1 Hz (um pulso por segundo) a partir de um clock de 50 MHz, o

valor de comparação deverá ser 50.000.000 / 2 = 25.000.000, o que exigirá registradores, muxes e comparadores de 25 bits (2^{25} = 33.554.432, suficiente, portanto para contar até 25.000.000).

3. Implementação em VHDL

A Figura 2 a seguir mostra o código fonte em VHDL que implementa o mesmo circuito mostrado anteriormente. Nesse caso foi utilizada a palavra-chave GENERIC para parametrizar o valor do divisor (que, como visto, deve ser a metade do valor de divisão desejado) e, por tabela,a quantidade de bits do contador/mux/comparador por meio da palavra-chave RANGE. Esse circuito pode ser usado como um componente básico em outro circuito em VHDL. Embora o valor do divisor foi fixado em 5 no circuito, esse valor pode ser sobrescrito pelo circuito de nível superior que venha a utilizá-lo, por meio da diretiva GENERIC MAP, possibilitando a parametrização total do divisor.

```
13  library ieee;
14    use ieee.std_logic_1164.all;
15
16  entity divider is
17    generic(divisor: integer := 5);
18    port(
19      clk_in      : in  std_logic;    -- clock in.
20      clk_out     : out std_logic     -- clock in dividido 2 x divisor
21      );
22  end divider;
23  ---------------------------------------------------------------------
24  architecture divisor of divider is
25  signal temp: std_logic :='0';
26  begin
27
28      P_div:process (clk_in)
29      variable count:integer range 0 to divisor;
30      begin
31          if clk_in'event and clk_in = '1' then
32              count := count + 1;
33              if (count = divisor) then
34                  temp <= not temp;
35                  count := 0;
36              end if;
37           end if;
38      end process P_div;
39      clk_out <= temp;
40
41  end divisor;
```

Figura 2: Código fonte VHDL para divisor digital síncrono.

Observe o uso de ***variable*** em vez de ***signal*** dentro do process ***P_div***. A diferença entre ***signal*** e ***variable*** será discutida mais adiante, no item 7.

4. Simução no Quartus do divisor básico

A Figura 3 abaixo mostra o resultado de uma simulação no Quartus. O sinal de saída tem 1/10 da frequência do sinal de entrada e é síncrono com ele.

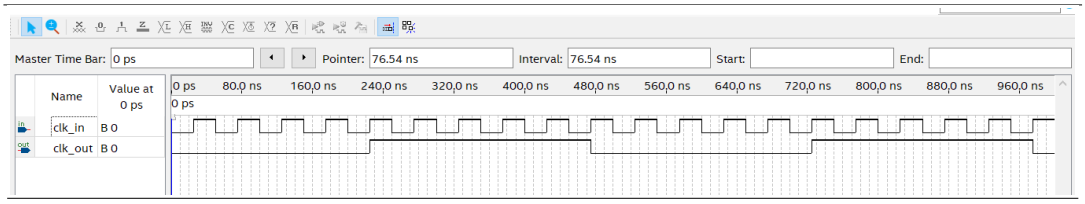

Figura 3: Resultado da simulação no Quartus – clk_out = clk_in / 10.

Tanto o circuito no LogiSim-evolution, como o correspondente projeto no Quartus estão disponíveis no arquivo ***divider.zip*** que acompanha este tutorial.

5. Divisor com múltiplas saídas

Quando mais de um divisor é necessário no circuito, se houver uma relação de multiplicidade entre as saídas, pode-se proceder à divisão em cadeia, dividindo o sinal já dividido mais uma vez. O circuito da Figura 4 a seguir, mostra o caso em que o sinal de habilitação do divisor por 10 anterior (que, como foi visto, ocorre a cada 5 CLK_IN), é utilizado para habilitar um divisor por 3, de maneira que o CLK_OUT_2 tem 1/30 da frequência de CLK_IN.

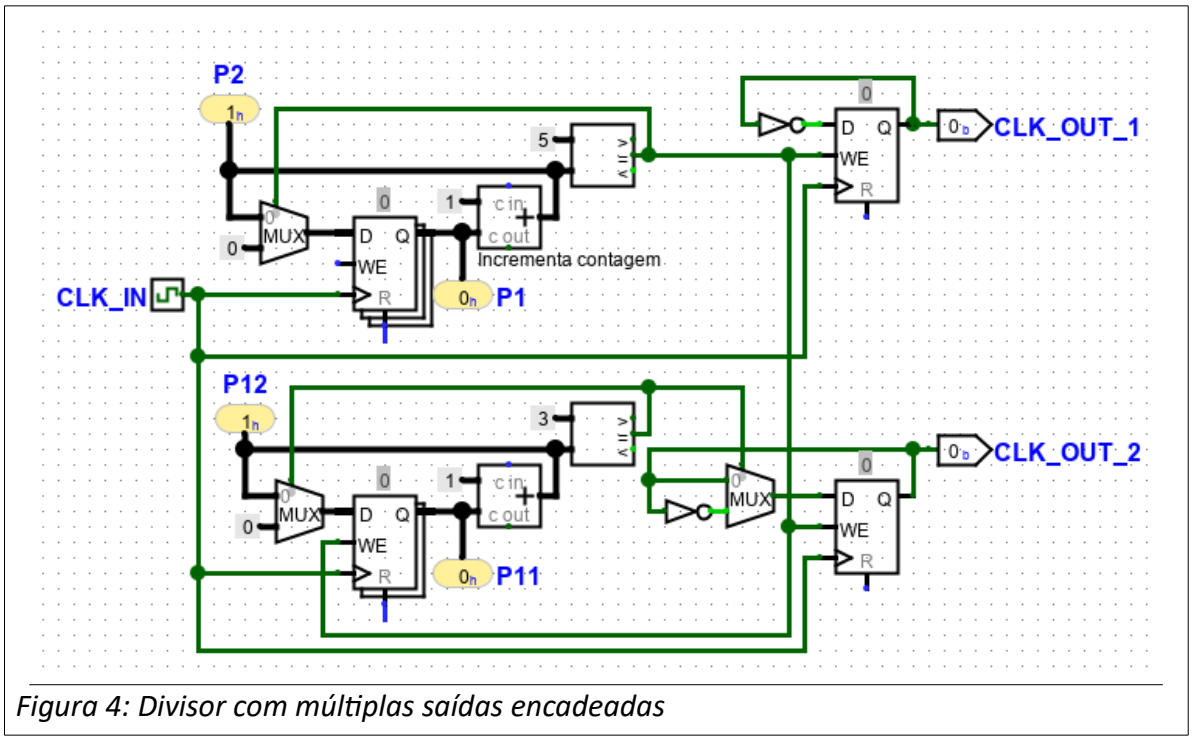

Figura 4: Divisor com múltiplas saídas encadeadas

Note-se, também, que o circuito da Figura 4 é totalmente síncrono. Ambos os sinais gerados na saída são síncronos com CLK_IN. Note-se, ainda, que, caso o divisor por 30 fosse implementado separadamente, como um divisor básico separado, seriam necessários 4 registradores em vez dos dois usados nesse esquema. Assim o encadeamento dos divisores propiciou uma economia de recursos.

A Figura 5 a seguir mostra a implementação desse circuito em linguagem VHDL. Note que a contagem do segundo contador foi implementada por um bloco ***if*** aninhado dentro do bloco ***if*** principal.

```
17  library ieee;
18    use ieee.std_logic_1164.all;
19
20  entity divider is
21    generic(divisor_1: integer := 5;
22            divisor_2: integer := 3);
23    port(
24      clk_in      : in  std_logic;    -- clock in.
25      clk_out_1   : out std_logic;    -- clock in dividido 2 x divisor_1
26      clk_out_2   : out std_logic     -- clock in dividido por 2 x divisor_1
        x divisor_2
27      );
28  end divider;
29  ---------------------------------------------------------------------------
    ---
30  architecture divisor of divider is
31  signal temp_1, temp_2: std_logic :='0';
32  begin
33
34      P_div:process (clk_in)
35      variable count_1:integer range 0 to divisor_1;
36      variable count_2:integer range 0 to divisor_2;
37      begin
38          if clk_in'event and clk_in = '1' then
39              count_1 := count_1 + 1;
40              if (count_1 = divisor_1) then
41                  temp_1 <= not temp_1;
42                  count_1 := 0;
43                  count_2 := count_2 + 1;
44                  if(count_2 = divisor_2) then
45                      count_2 := 0;
46                      temp_2 <= not temp_2;
47                  end if;
48              end if;
49           end if;
50      end process P_div;
51      clk_out_1 <= temp_1;
52      clk_out_2 <= temp_2;
53
54  end divisor;
```

Figura 5: Descrição VHDL do divisor digital síncrono com saídas encadeadas.

6. Simulação no Quartus do divisor com saídas encadeadas

A Figura 6 a seguir mostra o resultado da simulação no Quartus do circuito descrito em linguagem VHDL da Figura 5. Na simulação percebe-se que ambas as saídas são síncronas com CLK_IN.

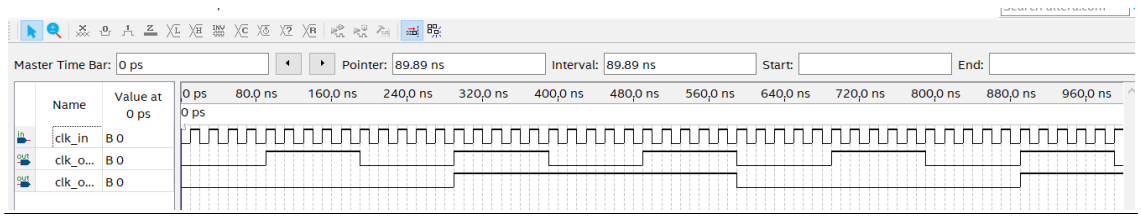

Figura 6: Resulteado da simulação do divisor digital síncrono com saídas encadeadas.

Tanto o projeto para LogiSim-evolution como para o Quartus para o divisor com saídas encadeadas estão disponíveis no arquivo ***divider.zip*** que acompanha este tutorial.

7. Diferença entre *variable* e *signal* em VHDL

Nos circuitos anteriores, na parte de lógica sequencial, caracterizada por um bloco *process*, foram utilizadas *variable's* em vez de *signal's*. As principais diferenças são:

- Escopo: Uma *variable* é visível apenas dentro do *process* no qual foi declarada. Um *signal* é visível dentro da *architecture* onde foi declarado.
- Sinal de atribuição: Valores devem ser atribuídos a uma *variable* usando o sinal **:=** enquanto que atribuições a um *signal* são feitas com o sinal **<=**.
- Talvez a diferença mais importante é que uma *variable* modificada dentro de um *process* tem seu valor atualizado **imediatamente**, enquanto que um *signal* só será atualizado quando o *process* for concluído, ou seja, na próxima transição de sinal do *process*.

Para melhor entender essa última afirmação, foi elaborado um projeto com dois *process*, praticamente idênticos, exceto pelo fato de um deles utilizar *variable* e o outro utilizar *signal*. A listagem em VHDL está na Figura 7 a seguir.

```
19  library ieee;
20    use ieee.std_logic_1164.all;
21
22  entity divider_sig_var is
23    generic(divisor: integer := 5);
24    port(
25      clk_in      : in  std_logic;    -- clock in.
26      clk_out_var : out std_logic;    -- clock in dividido 2 x divisor - usa var
27      clk_out_sig : out std_logic     -- clock in dividido 2 x divisor - usa sig
28      );
29  end divider_sig_var;
30  ---------------------------------------------------------------------------
31  architecture divisor of divider_sig_var is
32  signal temp_var: std_logic :='0';               -- usada no proces com variable
33  signal temp_sig: std_logic :='0';               -- usada no processo com signal
34  signal count_sig: integer range 0 to divisor;   -- usada no processo com signal
35  begin
36  --================== Processo usando variable: ================================
37      P_div_var:process (clk_in)
38      variable count_var: integer range 0 to divisor;
39      begin
40          if clk_in'event and clk_in = '1' then
41              count_var := count_var + 1;
42              if (count_var = divisor) then
43                  temp_var <= not temp_var;
44                  count_var := 0;
45              end if;
46           end if;
47      end process P_div_var;
48      clk_out_var <= temp_var;
49  --=================== Processo usando signal: ================================
50      P_div_sig:process (clk_in)
51      begin
52          if clk_in'event and clk_in = '1' then
53              count_sig <= count_sig + 1;
54              if (count_sig = divisor) then
55                  temp_sig <= not temp_sig;
56                  count_sig <= 0;
57              end if;
58           end if;
59      end process P_div_sig;
60      clk_out_sig <= temp_sig;
61
62  end divisor;
63
```

Figura 7: Circuito para evidenciar diferenças entre signal e variable.

No circuito da Figura 7 acima, o *process P_div_var* faz a contagem por meio de uma *variable*

(*count_var* declarada dentro do *process*, na linha 38), enquanto que o *process P_div_sig* faz a mesma contagem por meio de um *signal* (*count_sig* declarado fora do *process* na linha 14).

O resultado é que o valor incrementado de *count_sig* só é atualizado quando o *process P_div_sig* é concluído, ou seja, somente na próxima transição de CLK_IN, o que faz com que *count_sig* conte seis períodos de CLK_IN, enquanto que *count_var*, que é atualizado imediatamente, dentro de *P_div_var*, conte corretamente 5 períodos de CLK_IN.

O resultado da simulação desse circuito pode ser visto na Figura 8 a seguir.

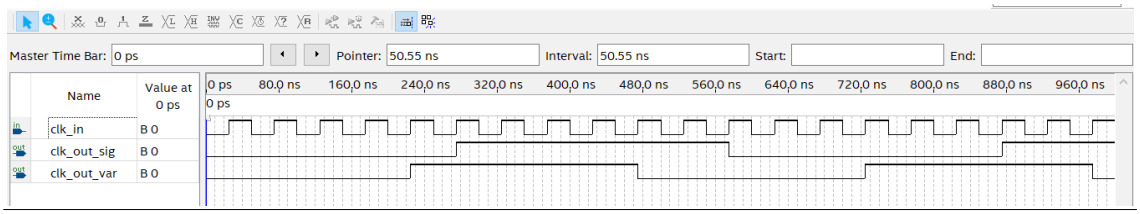

Figura 8: Resultado da simulação de comparação entre variable e signal

Como é possível notar na figura, *clk_out_sig* é atualizada sempre com um período a mais de CLK_IN, o que faz com que tenhamos um divisor por 12 em vez de um divisor por 10 como desejado originalmente.

O circuito implementado no LogiSim-evolution na Figura 9 abaixo ilustra a diferença de conexão realizada por um *signal* e por uma *variable*.

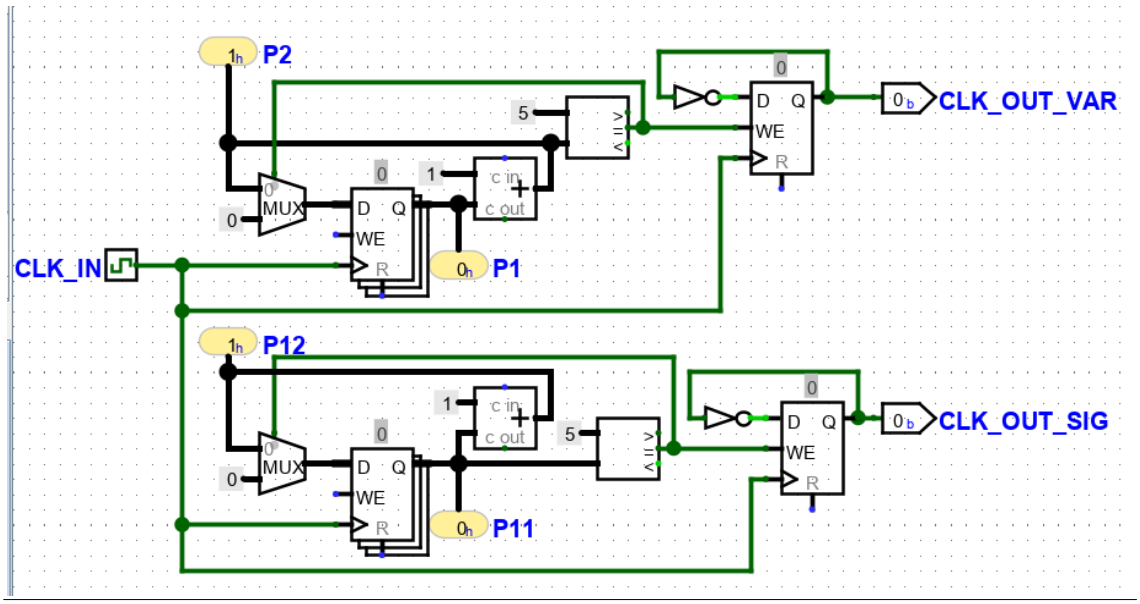

Figura 9: Diferença entre variable e signal no LogiSim-evolution

Na metade superior da figura percebe-se que quando o sinal incrementado é declarado como *variable* (em VHDL), ele está imediatamente disponível para o comparador. Já na metade inferior, onde o sinal incrementado foi declarado (em VHDL) como *signal*, seu valor incrementado só estará disponível para o comparador na próxima transição de CLK_IN.

Outra maneira de interpretar essa diferença é considerar que sinais declarados como *variable* estão

na entrada dos registradores do *process*, enquanto que sinais declarados como *signal* estão *na saída* dos registradores do *process*.

Tanto o projeto para o LogiSim-evolution, como para o Quartus estão disponíveis no arquivo ***divider.-zip*** que acompanha este tutorial.